PROYECTO FINAL DEL CURSO

> **Sistema Inteligente de Análisis y Procesamiento de Streams de Datos en Tiempo Real para Detección de Patrones y Recomendaciones**

INFORMACIÓN GENERAL

Curso: Estructuras de Datos y Algoritmos
Programa: Postgrado en Ciencia de Datos e Inteligencia Artificial
Modalidad: Equipos de 4-5 estudiantes
Fecha de entrega: *Por definir* **Peso en la nota final:** 40%

CONTEXTO Y MOTIVACIÓN

En el mundo actual, las empresas procesan billones de eventos diarios: transacciones financieras, interacciones en redes sociales, logs de servidores, señales de IoT, entre otros. La capacidad de procesar estos datos en **tiempo real** con **eficiencia algorítmica** es crítica para:

- Detectar fraudes en milisegundos
- Generar recomendaciones personalizadas al instante
- Identificar anomalías en sistemas críticos
- Optimizar recursos computacionales

Este proyecto simula un escenario real donde deben diseñar e implementar un sistema completo que combine múltiples estructuras de datos avanzadas para resolver problemas complejos de forma eficiente.

OBJETIVO GENERAL

Diseñar, implementar y evaluar un **sistema de procesamiento de streams de datos** que integre estructuras de datos avanzadas (Sparse Matrix, LSH, Bloom Filters, Tries, Skip Lists, B+Tree, R-Tree, KD-Tree, M-Tree, etc.) para resolver problemas de:

1. **Procesamiento con prioridades**
2. **Búsquedas ultra-rápidas**
3. **Detección de duplicados a escala**
4. **Recomendaciones en tiempo real**
5. **Análisis de patrones textuales**

ESCENARIO DEL PROYECTO

Opción A: Plataforma de Streaming de Video (Tipo Netflix/YouTube)

Dataset: 10 millones de eventos de usuarios interactuando con contenido

Eventos incluyen:

- Inicio de reproducción
- Pausa/Reanudación
- Cambio de calidad
- Búsqueda de contenido
- Valoración (like/dislike)
- Comentarios

Requisitos del sistema:

1. **Cola de Procesamiento con Prioridades**
    - Usuarios Premium tienen prioridad en procesamiento
    - Eventos de pago (compras, suscripciones) tienen máxima prioridad
2. **Caché de Contenido Inteligente**
    - Mantener los 1,000 videos más accedidos en caché
        - Política de evicción: LRU (Least Recently Used)
3. **Detección de Usuarios Duplicados/Bots**
    - Identificar patrones sospechosos (mismo usuario, múltiples IPs)
4. **Sistema de Recomendación**
    - Top-K videos similares basado en co-watching patterns
    - Calcular similaridad entre usuarios
5. **Autocompletado de Búsquedas**
    - Sugerencias instantáneas mientras el usuario escribe
    - Ranking de sugerencias por popularidad

Opción B: Sistema de Comercio Electrónico (Tipo Amazon/Mercado Libre)

Dataset: 10 millones de transacciones y eventos de usuarios

Eventos incluyen:

- Búsquedas de productos
- Vistas de productos
- Agregados al carrito
- Compras completadas
- Devoluciones
- Reviews

Requisitos del sistema:

1. **Sistema de Procesamiento de Órdenes**
    - Órdenes express (priority alta)
    - Órdenes estándar (priority media)
    - Órdenes programadas (priority baja)

2. **Índice de Productos para Búsqueda Rápida**
    - Búsqueda por SKU, nombre, categoría
    - Tiempo de búsqueda objetivo: O(1)
3. **Detección de Fraude en Tiempo Real**
    - Detectar patrones sospechosos (misma tarjeta, múltiples intentos)
    - Ventana temporal deslizante
4. **Top-N Productos Más Vendidos por Categoría**
    - Mantener ranking en tiempo real
    - Investigar sobre KNN con **índices multdimensionales**
    - Actualización incremental eficiente
5. **Autocompletado de Búsqueda de Productos**
    - Sugerencias ponderadas por ventas
    - Actualización dinámica de pesos

Opción C: Sistema de Análisis de Redes Sociales (Tipo Twitter/LinkedIn)

Dataset: 10 millones de posts, interacciones y menciones

Eventos incluyen:

- Posts/Tweets
- Likes/Retweets
- Comentarios
- Menciones
- Seguimientos
- Búsquedas de hashtags

Requisitos del sistema:

1. **Feed de Noticias Personalizado**
    - Combinar posts de múltiples fuentes con prioridad
    - Prioridad por engagement, relevancia temporal, conexión
2. **Trending Topics en Tiempo Real**
    - Top-K hashtags más mencionados (ventana de 1 hora)
    - Actualización continua
3. **Detección de Usuarios Similares (Who to Follow)**
    - K vecinos más cercanos basado en intereses
    - Investigar e implementar **MinHash y LSH**
4. **Índice de Búsqueda de Contenido**
    - Búsqueda de posts por palabras clave
    - Ranking por relevancia (TF-IDF simplificado)
5. **Caché de Consultas Frecuentes**

- Almacenar resultados de búsquedas populares
- Investigar **LRU Cache**
- Manejo de colisiones

COMPONENTES OBLIGATORIOS DEL PROYECTO

1. INVESTIGACIÓN Y DOCUMENTACIÓN (40%)

Deben investigar, comprender e implementar **AL MENOS 3** de las siguientes estructuras avanzadas:

Estructuras Avanzadas a Investigar:

A. Para Datos Masivos:

- **Bloom Filter**: Detección probabilística de membresía
- **Count-Min Sketch**: Estimación de frecuencias en streams
- **HyperLogLog**: Conteo de elementos únicos
- **Cuckoo Hashing**: Hash table sin colisiones encadenadas

B. Para Búsquedas Geométricas/Espaciales:

- **KD-Tree**: Búsquedas en espacios multidimensionales
- **R-Tree**: Índices espaciales para datos geográficos
- **Quadtree/Octree**: Particionamiento espacial

C. Para Texto y Cadenas:

- **Trie (Prefix Tree)**: Búsquedas de prefijos
- **Suffix Tree/Suffix Array**: Búsquedas de patrones
- **Aho-Corasick**: Búsqueda de múltiples patrones

D. Para Similitud y Búsqueda Aproximada:

- **LSH (Locality Sensitive Hashing)**: Búsqueda de vecinos aproximados
- **MinHash**: Estimación de similaridad de conjuntos
- **SimHash**: Detección de documentos similares

E. Para Persistencia y Bases de Datos:

- **B-Tree/B+ Tree**: Índices de bases de datos
- **LSM Tree (Log-Structured Merge Tree)**: Bases de datos modernas
- **Skip List**: Alternativa probabilística a árboles balanceados

F. Para Datos Dispersos:

- **Sparse Matrix con Hash Table**: Matrices con mayoría de ceros
- **Compressed Sparse Row (CSR)**: Almacenamiento eficiente

G. Para Escalabilidad:

- **Extendible Hashing**: Hash dinámico para bases de datos
- **Linear Hashing**: Crecimiento gradual de hash tables
- **Consistent Hashing**: Distribución en sistemas distribuidos

Requisitos de Investigación:

Para cada estructura avanzada elegida, documentar:

1. **Fundamento teórico**
   - ¿Qué problema resuelve?
   - ¿Cómo funciona internamente?
   - Complejidad temporal y espacial
2. **Casos de uso reales**
   - ¿En qué contextos es óptima?
   - ¿Cuándo NO usarla?
3. **Trade-offs**
   - Ventajas vs estructuras tradicionales
   - Limitaciones y desventajas
   - Consumo de memoria vs velocidad
4. **Referencias académicas**
   - Paper original (si existe)
   - Implementaciones de referencia
   - Benchmarks publicados

2. IMPLEMENTACIÓN DEL SISTEMA (20%)

Procurar realizar una implementación que evidencie el uso correcto de las estructuras de datos avanzadas. La implementación estará centrada en la demostración empírica de la eficiencia de las estructura de datos y también en la visualización de resultados esperados.

Funcionalidades Mínimas:

- Procesamiento de Streams con Prioridades
- Sistema de Caché Inteligente
- Búsqueda Ultra-Rápida
- Detección de Duplicados/Anomalías
- Top-K Tracker en Tiempo Real

3. EXPERIMENTACIÓN Y ANÁLISIS (25%)

3.1 Análisis de Complejidad Empírica

Para cada operación crítica:

1. **Generar datasets de diferentes tamaños**
   - 1K, 10K, 100K, 1M, 10M elementos
2. **Medir tiempo de ejecución**
   - Repetir 100 veces y promediar
   - Registrar desviación estándar

3. **Ajustar curva teórica**
    - O(1): Tiempo constante
    - O(log n): Logarítmica
    - O(n): Lineal
    - O(n log n): Lineal-logarítmica
4. **Visualizar resultados**

```
import matplotlib.pyplot as plt

# Gráfico log-log para identificar complejidad
plt.loglog(sizes, times, 'o-', label='Empírico')
plt.loglog(sizes, theoretical_O_logn, '--', label='O(log n) teórico')
plt.xlabel('Tamaño del dataset (n)')
plt.ylabel('Tiempo (ms)')
plt.legend()
plt.title('Análisis de Complejidad: Heap Insert')
```

3.2 Análisis de Memoria

```
import tracemalloc

# Medir consumo de memoria de cada estructura
tracemalloc.start()

# Insertar 1M elementos
for i in range(1000000):
    structure.insert(data[i])

current, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()

print(f"Memoria actual: {current / 10**6:.2f} MB")
print(f"Memoria pico: {peak / 10**6:.2f} MB")
```

Ejemplos de comparación:

- Bloom Filter vs Hash Set (para detección)
- Trie vs Hash Table (para prefijos)
- Sparse Matrix vs Dense Array

4. PRESENTACIÓN E INFORME (15%)

4.1 Presentación Interactiva

El grupo debe demostrar el manejo de las estructura de datos, explicarlas interactivamente y responder acertivamente preguntas del docente y de sus compañeros.

Es importante crear visualizaciones interactivas o diapositivas que demuestren:

A. Comparativas de Estructuras

```
# Gráficos comparativos:
# - Barras: Tiempo por operación (diferentes estructuras)
# - Líneas: Escalabilidad (tamaño vs tiempo)
# - Scatter: Memoria vs Velocidad (trade-offs)
```

B. Análisis de Resultados

```
# Ejemplo de visualización:
# - Top-K productos/videos/hashtags
# - Tasa de aciertos del caché (hit rate)
```

4.2 Informe Técnico

Estructura del documento (8-12 páginas):

1. Resumen Ejecutivo (1 página)

- Problema abordado
- Solución propuesta
- Resultados principales

2. Introducción y Contexto (1-2 páginas)

- Descripción del escenario elegido
- Justificación de estructuras seleccionadas
- Objetivos específicos del proyecto

3. Investigación de Estructuras Avanzadas (3-5 páginas)

- Por cada estructura avanzada:
  - Fundamento teórico
  - Paper original y referencias
  - Análisis de complejidad algoritmica (tiempo y espacio)
  - Casos de uso reales
  - Trade-offs

4. Diseño e Implementació del Sistema (1-2 páginas)

- Arquitectura general
- Flujo de datos
- Decisiones de diseño y justificación
- Detalles técnicos relevantes
- Desafíos encontrados y soluciones

6. Experimentación y Resultados (1-2 páginas)

- Metodología de benchmarking
- Datasets de prueba
- Resultados empíricos con gráficos
- Verificación de complejidades teóricas
- Análisis de memoria

- Comparativas con alternativas

7. Conclusiones y Trabajo Futuro (1 página)

- Logros principales
- Lecciones aprendidas
- Limitaciones identificadas
- Posibles mejoras y extensiones

8. Referencias

- Papers académicos
- Documentación técnica
- Repositorios consultados

ENTREGABLES

1. Repositorio de GitHub (Obligatorio)

Debe incluir:

- Código fuente completo
- Documentación (README e Informe)
- Notebooks de experimentación

README.md debe contener:

```
# [Nombre del Proyecto]

## Integrantes
- Estudiante 1 (email)
- Estudiante 2 (email)
- Estudiante 3 (email)

## Descripción
[Breve descripción del proyecto]

## Estructuras Avanzadas Implementadas
1. Bloom Filter - [Breve descripción]
2. Trie - [Breve descripción]
3. LSH - [Breve descripción]

## Instalación

pip install -r requirements.txt

## Uso

python src/main.py --dataset data/dataset_10M.csv

## Benchmarks
[Cómo reproducir los benchmarks]

## Resultados Principales
[Resumen de resultados con gráficos]

## Referencias
[Lista de papers y recursos]
```

2. Informe Técnico (PDF)

Formato: PDF, 8-12 páginas

Nombre: `Proyecto_Final_[NombreGrupo].pdf`

Subir a: Campus virtual + incluir en repositorio

4. Presentación Final (Obligatorio)

Fecha: Última semana de clases

Duración: 15 minutos + 5 minutos de preguntas

Slides: 10-15 diapositivas

Contenido:

- Problema y solución (2 slides)
- Estructuras avanzadas (4-5 slides)
- Demo en vivo (3-4 min)
- Resultados experimentales (2-3 slides)
- Conclusiones (1 slide)

Recursos de Investigación

Papers Fundamentales:

- Bloom, B. H. (1970). "Space/time trade-offs in hash coding with allowable errors"
- Cormode, G., & Muthukrishnan, S. (2005). "An improved data stream summary: the count-min sketch"
- Indyk, P., & Motwani, R. (1998). "Approximate nearest neighbors: towards removing the curse of dimensionality"
- ...